CSC 212: Data Structures and Abstractions

08: Priority Queues

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Spring 2026

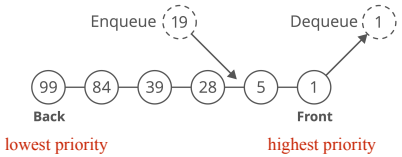
THINK BIG WE DO℠

Priority queues

Priority queues

- Definition
  - ✓ a **priority queue** is a data structure similar to a queue, but where each element has an associated priority
  - ✓ elements with **higher priority** are removed before elements with lower priority
- Operations
  - ✓ mutators
    - **enqueue**: (**push**) add element with an associated priority
    - **dequeue**: (**pop**) remove element with highest priority
  - ✓ accessors
    - **top**: returns a reference to the top (first) element
- Applications
  - ✓ algorithms for graphs, event-driven simulation, search methods in AI, job scheduling in OS, etc.

all core **queue** operations run in $\Theta(\log n)$ time (assuming proper implementations)



<key,value> pairs

- Elements in a priority queue can be implemented as a collection of **<key,value>** pairs
  - ✓ **key**: determines the priority, used for comparison
  - ✓ **value**: actual data/payload associated with that priority

Operations (min-pq)	Return value
enqueue(5, A)	
enqueue(10, D)	
enqueue(3, B)	
dequeue()	(3, B)
enqueue(7, C)	
dequeue()	(5, A)
dequeue()	(7, C)
size()	1
isEmpty()	FALSE

Practice

- What is the output of the following code?

```
#include <iostream>
#include <queue>

int main() {
    // by default, std::priority_queue is a max-priority queue
    std::priority_queue<std::pair<int, std::string>> pq;

    pq.push({3, “New York”});
    pq.push({9, "Boston"});
    pq.push({5, "Chicago"});

    while (!pq.empty()) {
        std::cout << pq.top().first << ": " << pq.top().second << "\n";
        pq.pop();
    }

    return 0;
}
```


Practice

- What is the output of this code?

```
#include <iostream>
#include <queue>
#include <utility> // for std::pair

int main() {
    // default priority_queue - max-priority queue behavior
    std::priority_queue<std::pair<int, std::string>> pq;

    pq.push({3, "Job 1"});
    pq.push({1, "Job 2"});
    pq.push({5, "Job 3"});
    pq.pop();
    pq.push({2, "Job 4"});
    pq.pop();
    pq.push({7, "Job 5"});
    pq.pop();
    pq.pop();
    pq.push({7, "Job 6"});
    pq.push({7, "Job 7"});

    while (! pq.empty()) {
        std::pair<int, std::string> top = pq.top();
        std::cout << top.second << std::endl;
        pq.pop();
    }

    return 0;
}
```


Implementation

- Using arrays
  - ✓ ensure `enqueue` and `dequeue` work efficiently
  - ✓ array can be fixed-length or a dynamic array
- Considerations
  - ✓ highest priority
    - in a **max-priority queue**, refers to largest key value
    - in a **min-priority queue**, refers to smallest key value
  - ✓ equal priority elements
    - in some implementations, these elements are served in FIFO order
    - in others, their order is undefined
  - ✓ throw an error when calling dequeue on an empty queue
  - ✓ throw an error when calling enqueue on a full queue

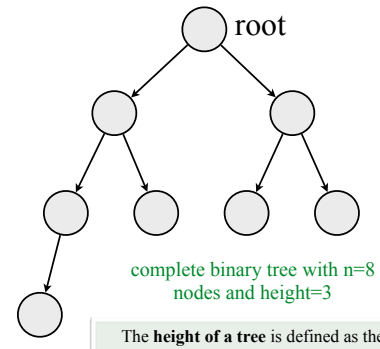
Implementation

- Array-based (keep an unsorted array)
  - ✓ enqueue at the end — $\Theta(1)$ cost (amortized cost if using a dynamic array)
  - ✓ dequeue (extract max/min) — $\Theta(n)$ cost
    - requires searching the entire array
- Array-based (keep a sorted array)
  - ✓ enqueue at position — $\Theta(n)$ cost
    - requires finding position for insertion and shifting elements
  - ✓ dequeue (extract max/min) — $\Theta(1)$ cost
- Binary heap (array-based)
  - ✓ most common and efficient
  - ✓ enqueue — $\Theta(\log n)$ cost
  - ✓ dequeue (extract max/min) — $\Theta(\log n)$ cost
  - ✓ can also build a binary heap from an unsorted array in $\Theta(n)$ cost (heapify)

Binary heaps

Complete binary tree

- Binary tree
  - ✓ tree data structure in which each **node** has at most two children, referred to as the left child and the right child
- Complete binary tree
  - ✓ binary tree in which every level, except possibly the last, is completely filled
    - all nodes in the last level are as far left as possible



complete binary tree with n=8 nodes and height=3

The **height of a tree** is defined as the number of edges on the longest path from the root to a leaf (terminal) node.

`The height of a complete binary tree with` n `nodes is` $\lfloor \log_2 n \rfloor$

Practice

- Consider a complete binary tree of height h
  - ✓ what is n_{max}, the max number of nodes in the tree as a function of h?
    - hint: use a summation formula
  - ✓ what is n_{min}, the min number of nodes in the tree as a function of h?

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$
$$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2}$$
$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$
$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2 = \frac{n^2(n+1)^2}{4}$$
$$\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}, c \neq 1$$

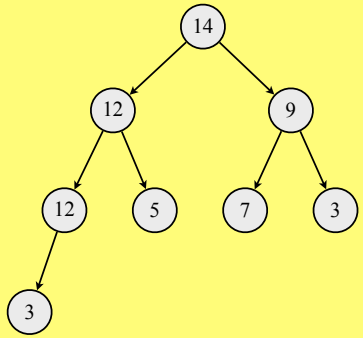
- For a complete binary tree of height h, the following holds: $n_{min} \leq n < n_{max} + 1$
  - ✓ take the logarithm (base 2) of this inequality and express h in terms of n

Binary heap

- Definition
  - ✓ **structure property**: a binary heap is a **complete binary tree**
  - ✓ **heap property**: a binary heap can be:
    - **max-heap**: each node's value is greater than or equal to its children's values
    - **min-heap**: each node's value is smaller than or equal to its children's values
- Considerations
  - ✓ the height of a binary heap is $\lfloor \log_2 n \rfloor$
  - ✓ the number of nodes at each level h is at most 2^h
  - ✓ the number of nodes in a heap is at most: $\sum_{i=0}^{h} 2^i = 2^{h+1} - 1$

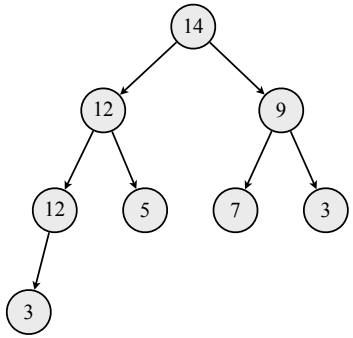
Practice

- Check:
  - ✓ structure property
  - ✓ heap-order property (max-heap)
- Add 3 elements at lowest level
  - ✓ without violating properties
- Change 2 values
  - ✓ that violate the heap property



Array representation

- A binary heap can be represented as an array
  - ✓ **root** is at index 0
  - ✓ **last element** is at index $n - 1$
- For any node at index i:
  - ✓ **left child** is at index $2i + 1$
  - ✓ **right child** is at index $2i + 2$
  - ✓ **parent** is at index $(i - 1)$ // 2



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
14	12	9	12	5	7	3	3								

`n=8, capacity=16`


```
class PriorityQueue {
    private:
        int *arr;
        size_t capacity;
        size_t size;

        size_t parent(size_t i) { return (i-1) / 2; }
        size_t left(size_t i) { return 2*i + 1; }
        size_t right(size_t i) { return 2*i + 2; }

        void upHeap(size_t i);
        void downHeap(size_t i);

    public:
        PriorityQueue(size_t cap) : capacity(cap), size(0) {
            arr = new int[capacity];
        }
        ~PriorityQueue() {
            delete [] arr;
        }

        void enqueue(const int& val);
        void dequeue();

        const int& top() const { return arr[0]; }
        size_t get_size() { return size; }
        size_t get_capacity() { return capacity; }
        bool empty() { return size == 0; }
};
```

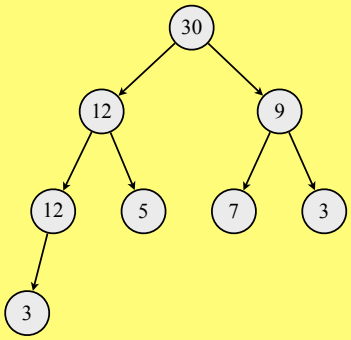

Enqueue (max-heap)

- Algorithm (min-heap is analogous)
  1. append element to the end of the array
  2. "bubble up" (a.k.a. `upHeap`)
     - **set** `idx` to the newly added element's position
     - **while** `idx` is not the root **and** element at `idx` is greater than its parent
       - swap them and update `idx` to the parent's position
- Time complexity
  - ✓ how many swaps are necessary in the worst case? $\Theta(\log n)$

https://visualgo.net/en/heap

Practice

- Enqueue 20
  - ✓ show resulting array
- Enqueue 1
  - ✓ show resulting array
- Enqueue 50
  - ✓ show resulting array



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
30	12	9	12	5	7	3	3								

size

Enqueue

```
void PriorityQueue::enqueue(const int& val) {
    if (size == capacity) {
        throw std::out_of_range("PriorityQueue is full");
    }
    arr[size] = val;
    size ++;
    upHeap(size-1);
}

void PriorityQueue::upHeap(size_t idx) {
    while (idx > 0) {
        size_t p = parent(idx);
        if (arr[idx] > arr[p]) {
            std::swap(arr[idx], arr[p]);
            idx = p;
        } else {
            break;
        }
    }
}
```

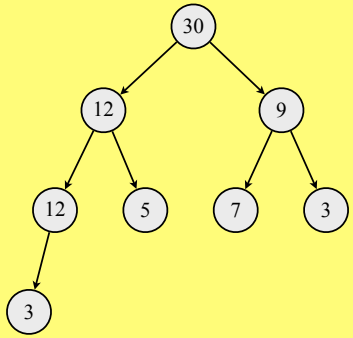

Dequeue (max-heap)

- Algorithm (min-heap is analogous)
  1. swap the root with the last element in the array
  2. remove the last element from the array (return this value if needed)
  3. "bubble down" (a.k.a. `downHeap`)
     - **set** `idx` to the root element's position
     - **while** `idx` is not a leaf **and** element at `idx` is smaller than its largest child
       - swap them and update `idx` to that child's position
- Time complexity
  - ✓ how many swaps are necessary in the worst case? $\Theta(\log n)$

https://visualgo.net/en/heap

Practice

- Dequeue
  - ✓ show resulting array
- Dequeue
  - ✓ show resulting array
- Dequeue
  - ✓ show resulting array



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
30	12	9	12	5	7	3	3								

size

Dequeue

```
void PriorityQueue::dequeue() {
    if (size == 0) {
        throw std::out_of_range("PriorityQueue is empty");
    }
    arr[0] = arr[size-1];
    size --;
    downHeap(0);
}

void PriorityQueue::downHeap(size_t i) {
    while (true) {
        size_t largest = i;
        size_t l = left(i);
        size_t r = right(i);
        if (l < size && arr[l] > arr[largest]) {
            largest = l;
        }
        if (r < size && arr[r] > arr[largest]) {
            largest = r;
        }
        if (largest != i) {
            std::swap(arr[i], arr[largest]);
            i = largest; // move down to largest
        } else {
            break;
        }
    }
}
```



```
int main() {
    PriorityQueue pq(100);

    pq.enqueue(3);
    pq.enqueue(1);
    pq.enqueue(5);
    pq.dequeue();
    pq.enqueue(2);
    pq.dequeue();
    pq.enqueue(7);
    pq.dequeue();
    pq.dequeue();
    pq.enqueue(7);
    pq.enqueue(7);

    while (! pq.empty()) {
        std::cout << pq.top() << std::endl;
        pq.dequeue();
    }

    return 0;
}
```


Performance (priority queues)

Method	Unsorted Array	Sorted Array	Binary Heap
Enqueue	O(1)	O(n)	O(log n)
Dequeue	O(n)	O(1)	O(log n)
Max/Min	O(n)	O(1)	O(1)
Size	O(1)	O(1)	O(1)
IsEmpty	O(1)	O(1)	O(1)

Practice

- Given a stream of integers, design an algorithm to efficiently report the **k-th largest element** seen so far at any time.
  - ✓ as new numbers arrive, your algorithm must update its data structure and return the current k-th largest number
- Sample input:
  - ✓ k = 3
  - ✓ values = {4, 5, 8, 2, 3, 5, 10, 9, 4}
- Output:
  - ✓ show k-th largest element after every insertion

Practice

- Given an array of integers and a window size k, design an algorithm that returns a new array where each element represents the **maximum value** within a sliding window of size k, as it moves from left to right across the input array

- Sample input:
  - ✓ k = 4
  - ✓ values = {1, 3, -1, -3, 5, 3, 6, 7}

- Output:
  - ✓ {3, 5, 5, 6, 7}

This sliding-window maximum is directly analogous to **max pooling** in neural networks